

useMemo

It is a powerful React hook used to memoize values, meaning it caches the result of a computation so that the computation doesn't get re-executed. This is especially useful for optimizing performance when dealing with expensive computations.

Syntax:

```
const memoizedValue = useMemo(() => {  
  // Expensive computation  
  return computedResult;  
}, [dependencies]);
```

Example Without useMemo (Less Optimized):

```
import React, { useEffect, useState } from 'react';  
  
function App() {  
  const [count, setCount] = useState(0);  
  function ExpensiveFunc(num) {  
    for (let i = 0; i < 10000000000; i++) {} // Simulate expensive computation  
    return num + 2;  
  }  
  useEffect(() => {  
    console.log(ExpensiveFunc(2)); // Computation runs every time 'count' changes  
  }, [count]);  
  return (  
    <div>  
      <h1>Count: {count}</h1>  
      <button onClick={() => setCount(count + 1)}>Increment</button>  
    </div>
```

```
);  
}
```

```
export default App;
```

The above example makes the program slower since the expensive computation is run every time the changes.

Example With useMemo (Optimized):

```
import React, { useEffect, useMemo, useState } from 'react'
```

```
function App() {
```

```
  const [count, setCount] = useState(0);
```

```
  const num = 2;
```

```
  const ExpensiveFunc = useMemo(() => {
```

```
    for (let i = 0; i < 10000000000; i++) {} // Simulate expensive computation
```

```
    return num + 2;
```

```
  }, [num]); // Computation runs only when 'num' changes
```

```
  useEffect(() => {
```

```
    console.log(ExpensiveFunc); // Logs the memoized computation result
```

```
  }, [count]); // Effect depends on 'count'
```

```
  return (
```

```
    <div>
```

```
      <h1>Count: {count}</h1>
```

```
      <button onClick={() => setCount(count + 1)}>Increment</button>
```

```
    </div>
```

```
  );
```

```
}
```

```
export default App;
```

unMemo optimizes the application by memoizing the function's output based on the dependencies.

useCallback

This hook in React is used to memoize functions so they don't get recreated unnecessarily during re-renders. This is useful when you have functions that depend on specific dependencies and you want to avoid triggering unnecessary computations or child component re-renders. It works similarly to `useMemo`, but for functions.

Syntax:

```
const memoizedCallback = useCallback(  
  () => {  
    // Your callback logic here  
  },  
  [dependencies]  
);
```

Example Without useCallback:

```
import React, { useState } from 'react';  
import UseRef_prac from './UseRef_prac';  
function App() {  
  const [count, setCount] = useState(0);  
  return (  
    <div>  
      <h1>Count: {count}</h1>  
      <button onClick={() => setCount(count + 1)}>Update</button>  
      <UseRef_prac />  
    </div>  
  );  
}
```

export default App;

In the example above, if the child component accepts a functional argument, it will re-render every time the parent re-renders due to the creation of a new function.

Example With useCallback:

```
import React, { useCallback, useState } from 'react';

import UseRef_prac from './UseRef_prac';

function App() {

  const [count, setCount] = useState(0);

  const memoizedHandler = useCallback(() => {

    console.log('Function memoized and reused');

  }, []);

  return (

    <div>

      <h1>Count: {count}</h1>

      <button onClick={() => setCount(count + 1)}>Update</button>

      <UseRef_prac handler={memoizedHandler} />

    </div>

  );

}

export default App;
```

Here, the function is memoized, preventing unnecessary re-renders.

useLayoutEffect

It is a React hook similar to `useEffect`, but it runs after DOM mutations and before the browser paints the screen, unlike `useEffect` which runs after the screen has been painted.

Syntax:

```
useLayoutEffect(function, [dependencies]);
```

Key Difference from `useEffect`:

1. `useLayoutEffect` ensures updates to the DOM happen before the screen is rendered to the user.
2. `useEffect` is better suited for operations that don't block the paint.

Custom Hooks

You can create custom React hooks to encapsulate reusable logic in your application. They allow you to abstract stateful logic, making your components cleaner and more efficient.

Steps to Create a Custom Hook:

1. Define a new function with a name starting with `use` (e.g., `useLocalStorage`).
2. Inside the function, utilize other hooks like `useState` or `useEffect`.
3. Return any values or functions you want to expose to other components.

Example:

```
import React from 'react';

import useLocalStorage from './useLocalStorage';

function App() {

  const [user, setUser] = useLocalStorage('user', '');
  const [id, setId] = useLocalStorage('id', '');

  return (

    <div>

      <input value={user} onChange={(e) => setUser(e.target.value)} />

      <h1>Hello {user}</h1>

      <input value={id} onChange={(e) => setId(e.target.value)} />

      <h1>ID: {id}</h1>

    </div>

  );
}

export default App;
```

Custom Hook Implementation:

```
import { useEffect, useState } from 'react';

function useLocalStorage(key, startValue) {

  const [storedValue, setStoredValue] = useState(

    localStorage.getItem(key) || startValue

  );

  useEffect(() => {

    localStorage.setItem(key, storedValue);

  }, [storedValue]);

  return [storedValue, setStoredValue];

}

export default useLocalStorage;
```